Knowledge-Based Systems 306 (2024) 112697

Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys

Bridging the gap between text-to-SQL research and real-world applications: A unified all-in-one framework for text-to-SQL

Mirae Han [a], Seongsik Park [a], Seulgi Kim [a], Harksoo Kim [b,*]

[a] Department of Artificial Intelligence, Konkuk University, 120 Neungdong-ro, Gwangjin-gu, Seoul, 05029, Republic of Korea
[b] Department of Computer Science and Engineering, Konkuk University, 120 Neungdong-ro, Gwangjin-gu, Seoul, 05029, Republic of Korea

ARTICLE INFO

Keywords:
Semantic parsing
Text-to-SQL
Table selection
Natural language processing

ABSTRACT

Existing text-to-SQL research assumes the availability of gold table when generating SQL queries. It is possible to effectively generate complex and difficult queries by leveraging information from the gold table. However, in real-world scenarios, determining which of the numerous tables in a database should be referenced is challenging. Therefore, existing models reveal a gap in achieving the core objective of *practicality* in text-to-SQL research. In response, we propose a practical framework that can effectively convert user questions into queries, even in scenarios where reference tables are not provided. By adding a phase to find tables, it can generate queries using only information from questions, mitigating the limitations that arise when restricting reference tables to a single one. We demonstrate that our methods are suitable for practical use in text-to-SQL systems by achieving performances comparable to those of existing models with simple structures.

1. Introduction

Recent search trends have shifted from keyword to query-based search. Query-based search enables comprehensive and personalized searches by understanding the intent and meaning inherent in user questions. Accordingly, the potential and applicability of the text-to-SQL task, which supports query-based searches in databases, are also gaining attention. Text-to-SQL is a task that converts natural language questions into SQL queries, reducing the necessity for specialized SQL knowledge in database searches. This allows non-experts to easily search for the information in databases using natural language. Fig. 1 illustrates the execution process of a text-to-SQL system. When a natural language question *"Which position does Loren Woods play?"* is given, the system extracts the column *position* and the value *Loren Woods*. Based on this information, the system generates a query that adheres to SQL syntax and corresponds to the meaning of the question. The query is then executed on the database, returning an answer to the question. In this way, text-to-SQL systems serve as interfaces between users and databases, supporting query-based searches on structured data.

The challenges of text-to-SQL primarily arise from the differences between heterogeneous data. Natural language and tables are based on different structures, and natural language and SQL follow distinct grammatical systems. Therefore, efforts are required to bridge these gaps. To address these challenges, existing works have improved schema linking, which connects unstructured natural language questions with structured table schemas, and decoding methods to generate grammatically correct and executable queries. However, these improvements have ultimately focused on enhancing model performance, often overlooking the intrinsic value of text-to-SQL: *practicality*. Most studies assume that both a natural language question and a gold table containing the answer to the question are provided as inputs. However, in real-world scenarios, users ask questions without prior knowledge of databases. They do not know which table contains the answer and are unable to identify which table to reference for converting their question into an SQL query. Therefore, the assumption that both the user question and gold table are provided as inputs is unrealistic, and existing models that rely on the gold table are expected to struggle in maintaining performance in a real-world setting.

Furthermore, existing models have the limitation of considering only the result derived from the gold table as the correct answer. Despite the absence of clear guidelines on table selection, these models generate results solely from the pre-determined gold table. However, when no explicit information about a particular table is mentioned in the question, it is reasonable to choose any table. As shown in Fig. 2, given a question *"When is Rebecca Ferguson's birthday?"*, existing systems return *19 October, 1983* from the pre-designated reference table *Actors* and consider it the only correct answer. In contrast, *21 July, 1986*, which can be derived from *Singers* table, is regarded as incorrect

* Corresponding author.
E-mail addresses: future26@konkuk.ac.kr (M. Han), a163912@konkuk.ac.kr (S. Park), cloudyju11@konkuk.ac.kr (S. Kim), nlpdrkim@konkuk.ac.kr (H. Kim).

https://doi.org/10.1016/j.knosys.2024.112697
Received 24 June 2024; Received in revised form 8 October 2024; Accepted 2 November 2024
Available online 12 November 2024
0950-7051/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

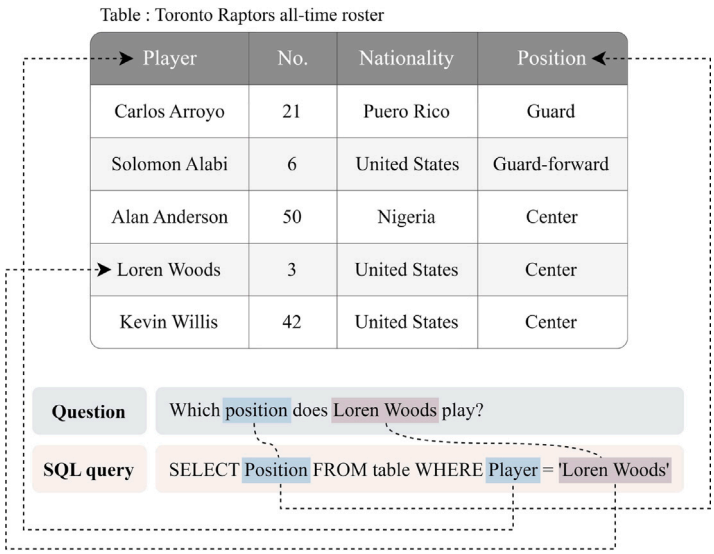

Fig. 1. The execution process of a text-to-SQL system.

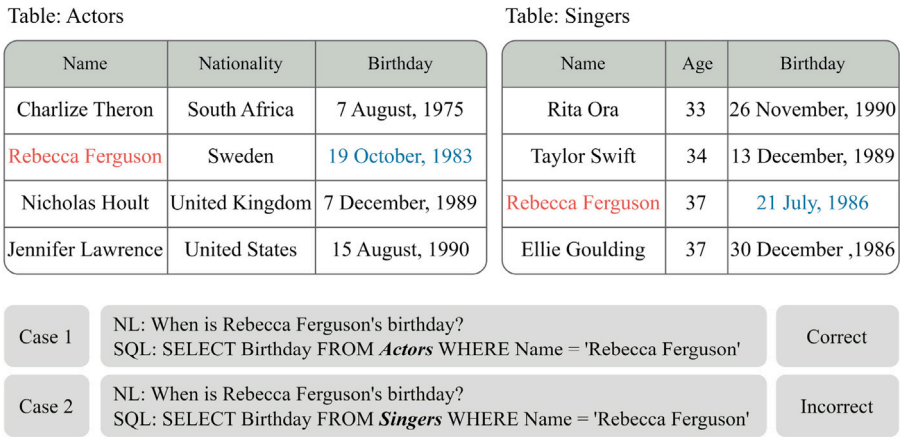

Fig. 2. Limitation of existing text-to-SQL models.

despite being a valid answer to the question. This demonstrates the limited range of answers that existing models are able to provide. Restricting answers arbitrarily is inappropriate in real-world scenarios. Therefore, when table information is not specified in the question, models should be able to generate a query after exploring all the tables. When there are multiple tables that can provide suitable answers to a question, the model should determine the table that best aligns with the question, based on sound reasons.

Table selection is essential for applying text-to-SQL systems in real-world scenarios. Accordingly, we propose a unified all-in-one framework for text-to-SQL, called uniText2SQL. It effectively performs table selection and SQL query generation, ensuring high practicality. Our framework consists of the table word extraction, table selection, and SQL query generation phases. In the table word extraction phase, we extract semantically important table words from a question, and in the table selection phase, we use these words to identify the tables within the database that contain the answer to the question. In the SQL query generation phase, we generate an SQL query by referencing the selected tables. While we introduce additional phases for table selection, these phases are not confined solely to table selection. The information extracted in the table word extraction phase not only serves as key clues for selecting tables but also plays a role in strengthening schema linking in the SQL query generation phase. The information obtained at each phase complements the subsequent phases, contributing to overall performance improvement.

The proposed framework supports query-based searches for users without SQL knowledge. It takes only the user question as the initial input, without requiring direct information about the database, and incorporates an auxiliary module for synonym processing to ensure practicality in real-world scenarios. Additionally, it expands the limited range of answers in existing systems caused by a fixed gold table. Instead of selecting a single table at once, we apply a criterion at each phase to evaluate the appropriateness of tables. This allows for the flexible selection of the most suitable tables according to the context of the question. When generating SQL queries, we consider both performance and efficiency. The proposed framework is structured as a pipeline model, making it convenient to maintain and improve. Each module performs an independent function, allowing targeted modifications in case of issues, thus making it suitable for real-world applications.

2. Related work

Text-to-SQL is a system that converts natural language questions into SQL queries and has developed with the goal of generating accurate queries based on a given gold table. Early works use WikiSQL [1], which consists of relatively simple query generation tasks that reference a single table. As text-to-SQL research advanced, the need for a dataset capable of addressing more complex problems emerged, leading to the introduction of Spider [2]. It includes operations such

as JOIN, GROUP BY, and ORDER BY, etc., requiring the ability to understand relationships between tables and handle complex query structures. Thus, text-to-SQL research has evolved toward enhancing system performance, focusing on solving complex query problems.

Early text-to-SQL systems using WikiSQL are based on sequence-to-sequence models [1,3]. These methods exhibit a limitation in that they cannot handle the conditions of the WHERE clause independently, as they process input and output sequentially. To address this, SQLNet [4] and TypeSQL [5] introduce sketch-based decoding and predict query components without relying on their order. Subsequently, new decoding methods were proposed to improve the accuracy and executability of SQL queries [6–8]. They either decode query components more systematically or adjust the decoding process based on the execution results of the query. Additionally, to facilitate a comprehensive understanding of both the question and the table, SQLova [9], X-SQL [10], and HydraNet [11] leverage pre-trained language models (PLMs). SQLova proposes a BERT-based [12] table-aware encoder to enable a more accurate comprehension of the question based on table information. X-SQL applies MT-DNN [13] and enhances not only the contextual information of the question but also the representation of structured data. HydraNet designs an encoder that aligns more coherently with the pre-training approach of BERT, improving the understanding of relationships between the question and the columns.

Existing works using Spider introduce more sophisticated model architectures to address complex text-to-SQL problems. In particular, various structural enhancements have been made to more effectively handle different data types, such as natural language, SQL, and table schemas. SyntaxSQLNet [14], SmBOP [15], and RESDSQL [16] propose decoding methods that are better suited to the structure of SQL, enabling the generation of queries that reflect the grammatical structure of SQL. IRNet [17] and NatSQL [18] mitigate the structural discrepancies between natural language and SQL through an intermediate representation, simplifying the conversion from natural language to SQL.

Along with this, the importance of schema linking has gradually emerged [19,20]. The difficulties in schema linking arise from structural differences between modalities, and methods for alleviating this challenge have been consistently proposed. Initial works that attempted schema linking adopted heuristic approaches [5,17], but subsequent works integrate learnable components to strengthen schema linking [21–25]. Bogin et al. [26] is the first work to encode tables using graph neural networks (GNNs) [27], emphasizing the importance of reflecting the structural characteristics of the schema. This has led to continuous development of GNN-based encoders. Various methods have been proposed to learn the interactions between a question and a table more profoundly and to maximize the utilization of structural information from both modalities.

These existing works have significantly improved the performance of text-to-SQL systems. However, there are still limitations in applying them to real-world environments. In research settings, a gold table is provided, and SQL queries are generated based on it. However, in real-world settings, only the user question is provided as input, so a module is required to automatically find the appropriate reference table relevant to the question. A system that retrieves pertinent information in response to a user question is a text retrieval system, which has evolved from heuristic approaches to probabilistic approaches, and to deep learning-based approaches. Early works analyze documents and questions at the word level and quantify the importance of each word to calculate the similarity between the two texts. Probabilistic models statistically analyze the word patterns between documents and questions to evaluate the their relevance probabilistically [28,29]. With the advent of deep learning [30], methods for representing documents and questions as dense vectors have been proposed [31,32], enabling searches based on deeper semantic relevance rather than simple word matching. They do not depend on simple word-level similarity but learn semantic features through dense vectors to evaluate intrinsic similarity. Furthermore, PLMs trained on large-scale text data capture complex semantic relationships between documents and questions more effectively, significantly enhancing the performance of text retrieval. There are text retrieval systems for table data. Herzig et al. [33] proposes the first dense retrieval-based method for table search. It maximizes table search performance by utilizing a specialized pre-training approach designed to effectively understand and process table data. In contrast, Wang et al. [34] emphasizes that a table-specific model is not essential for table search. Using DPR [32], they simplify the learning and inference processes while maintaining performance by understanding only the simple structure of tables.

Text retrieval is primarily applied to support downstream tasks such as question answering [32,35] and dialogue systems [36,37]. A text retriever searches for information relevant to the user question, and the results are utilized in downstream tasks. For the results of text retrieval to be smoothly integrated with downstream tasks, the tasks should not be studied independently but in an integrated manner. The simple combination of the two systems does not guarantee the accuracy and efficiency of each system, so structural improvements are necessary to mitigate error propagation between the systems and maintain efficiency. Oguz et al. [33] and Kostic et al. [38] indeed propose table retrieval methods effective for open-domain question answering. In contrast, there has been no research in the field of text-to-SQL that simultaneously addresses both table search and SQL generation. Therefore, we introduce a table selection module to enable the practical application of text-to-SQL systems in real-world environments, and propose a complementary framework that integrates table search and SQL generation.

3. Methods

3.1. Problem statement

The conventional text-to-SQL is defined as follows. We are given a training set of N examples $D_{train} = (x_i, t_i, q_i)$, where x_i is a natural language question, t_i is a table, and q_i is an SQL query which is the output of the text-to-SQL. Answer a_i is the value obtained by executing the SQL query q_i on the database. The goal of text-to-SQL is to train a model that, given a natural language question x and a table t, returns an SQL query q corresponding to question x, which eventually yields an answer a. However, we redefine text-to-SQL for practical applications as follows. Given only a natural language question x, we first find a set of tables T that are related to question x, and then leverage the information from the question x and table set T to generate an SQL query q corresponding to x. Finally, we execute query q to obtain answer a.

Our framework comprises three phases: table word extraction, table selection, and SQL query generation. Each phase is executed sequentially to generate an SQL query corresponding to the input question. Fig. 3 illustrates the overall process of the proposed framework. First, the table word extraction model extracts table words, such as columns and values, from the input question. Synonym processing is an auxiliary module that assists table word extraction. If the extracted words are synonyms of the terms used in the database, it replaces them with the corresponding database terms. Table set construction is an auxiliary module that assists table selection. It sorts the tables that contain the extracted table words and constructs a set of tables to be referenced for query generation. The table selection model selects the top k tables from this table set that are most relevant to the question. The SQL query generation model generates the final SQL query by comprehensively utilizing the information from the question and the k selected tables.

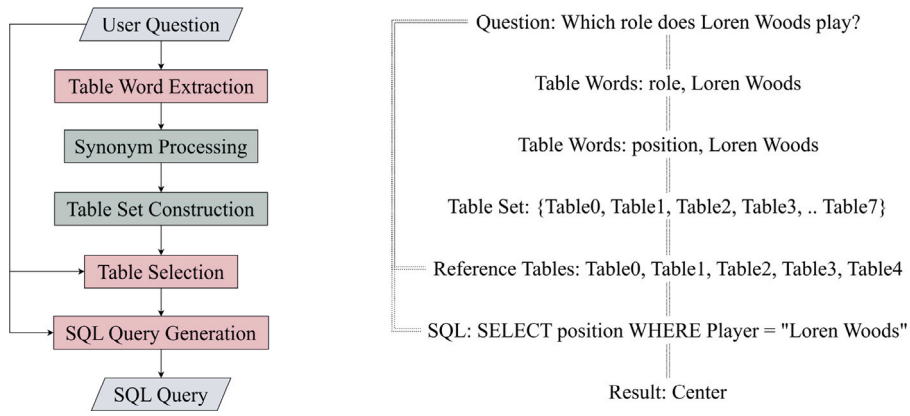

Fig. 3. A flowchart of the proposed framework. The flowchart on the left illustrates the process of generating an SQL query based on user question, while the structure diagram on the right represents the output values obtained through the execution of each phase. The final value in the structure diagram is the execution result of the generated query.

Country	Currency	Capital
Switzerland	Swiss Franc	Bern
the United States	United States Dollar	Washington, D.C
South Africa	South African Rand	Cape Town
South Korea	South Korean Won	Seoul
India	Indian Rupee	New Delhi

Case 1	NL: What is the capital of ***America***? SQL: SELECT Capital FROM table WHERE Country = '***America***'	No results
Case 2	NL: What is the capital of ***USA***? SQL: SELECT Capital FROM table WHERE Country = '***USA***'	No results
Case 3	NL: What is the capital of ***the United States***? SQL: SELECT Capital FROM table WHERE Country = '***the United States***'	Washington, D.C

Fig. 4. The necessity of handling synonyms.

3.2. Table word extraction

In the table word extraction phase, we extract semantically important table words from the natural language question. These words represent columns and values, and have a high relevance to the database. Table words are not only essential for understanding the meaning of the question, but they also play a crucial role in connecting the question and the database, which are two different types of data. Leveraging these characteristics, we use the extracted words to improve the performance of subsequent phases, table selection and SQL query generation. This differentiates our approach from existing works that use the extracted values merely as the values in the WHERE clause. In the table selection phase, we prioritize tables containing these words, thereby reducing the search space for the table selection model (Section 3.3). In the SQL query generation phase, we use these words to specify the table-related words in the question, thereby strengthening schema linking (Section 3.4). Finally, as in existing models, we use the extracted values as the values in the WHERE clause during the final query synthesis.

Additionally, the proposed framework employs an auxiliary module for synonym processing, which distinguishes it from existing models that do not address issues arising from the use of synonyms. In real-world scenarios, users are likely to use expressions that do not match the terms used in the database because they do not have prior knowledge of the database. This necessitates an additional module capable of handling synonyms. Fig. 4 illustrates the necessity of a synonym processing module. Users can ask about the capital of the United States in three different ways. Since *America*, *USA*, and *the United States* all refer to the same country, *Washington, D.C.* should be returned as the result, regardless of which expression is used. However, since only *the United States* is stored in the database, if *America* or *USA* is input, the model is unable to recognize *America* or *USA* in the question as the same entity as *the United States* in the database. As a result, schema linking fails, and if *America* or *USA* is used as the value in the WHERE clause, the final query will not return any result. This demonstrates that a synonym processing module is essential for applying text-to-SQL systems in real-world environments. When synonyms are included in the question, we need to replace them with the corresponding terms in the database. *America* and *USA* in the example should be substituted with *the United States*. At this phase, we employ a sequence labeling model to extract columns and values from the question, and use a trie algorithm to check for synonyms, which are then replaced with the corresponding terms from the database.

Fig. 5 illustrates the architecture of the sequence labeling model. The sequence labeling model sequentially processes the input sequence and predicts one label from the predefined label sets for each input token. In order to extract tokens corresponding to columns and values,

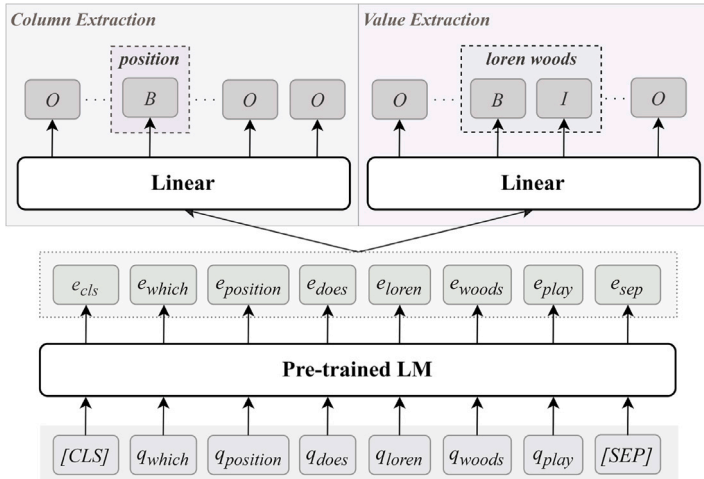

Fig. 5. The architecture of the sequence labeling model.

the model understands the context of the question and performs BIO (Beginning, Inside, Outside) tagging for each token. Tag B is assigned to tokens that are at the beginning of a column or value, and tag I is assigned to tokens that belong to a column or value but are not at the beginning. Tag O is assigned to tokens that are unrelated to columns or values.

The detailed process of extracting the columns and values through sequence labeling is as follows. First, the input question undergoes tokenization to be transformed into a form that can be processed by the PLM, BERT. It comprises multiple transformer layers, each of which captures the semantic relations between words and passes the information to the next layer, thereby yielding fixed-dimensional embedding vectors. Therefore, the tokenized question Q is input into BERT, which outputs the contextual embedding vectors E_q. The equations are as follows.

$$\begin{aligned} Q &= \{q_0, q_1, q_2, q_3, \dots, q_n\} \\ E_q &= PLM(Q) \\ E_q &= \{e_0, e_1, e_2, e_3, \dots, e_n\} \end{aligned} \tag{1}$$

After sentence embedding is completed, the labels for each token are predicted using contextualized vectors E_q. BIO labels are predicted using two independent linear layers, allowing for separate predictions of columns and values. The equations are as follows.

$$\begin{aligned} \hat{Y}_c &= W_c E_q + b_c \\ P(\hat{Y}_c|E_q) &= Softmax(\hat{Y}_c) \\ \hat{Y}_v &= W_v E_q + b_v \\ P(\hat{Y}_v|E_q) &= Softmax(\hat{Y}_v) \end{aligned} \tag{2}$$

W_c and b_c, W_v and b_v are trainable parameters for each layer. Each layer takes feature vectors E_q as the input and performs linear transformations. Linear transformation is the process of multiplying the weights by each element of the feature vectors and summing the results to produce $\hat{Y}_c$ and $\hat{Y}_v$. The outcomes of the linear transformations pass through the softmax function to produce the probability distributions $P(\hat{Y}_c|E_q)$ and $P(\hat{Y}_v|E_q)$ for each class. The class with the highest probability is assigned as the label for the token at that position. By applying independent layers to predict the columns and values, the two predictions can be performed simultaneously without affecting each other. As they represent different types of information, each linear layer captures distinct information and features to extract more suitable attributes for each type.

After extracting the columns and values through sequence labeling, we use the trie algorithm to discern the use of synonyms. If synonyms are found in a question, we replace them with words used in the database tables. The trie is a tree-like data structure that represents a set of strings and provides an efficient string search. For example, when looking up the word 'student' in a dictionary, we first locate the index of the first letter, 's'. Then we search for the letters 't', 'u', 'd', 'e', 'n', and 't' in order. The trie is a data structure in which this mechanism is logically applied. Trie is primarily used for storing strings and returns the associated data values using each string as a key.

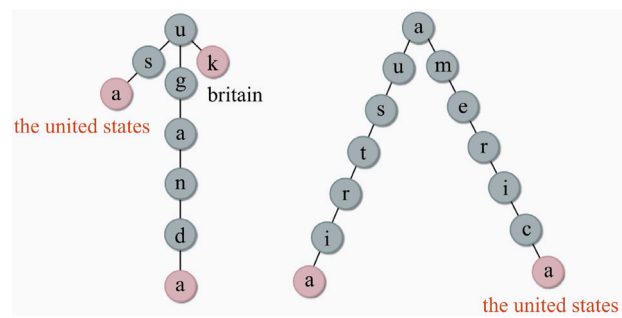

Fig. 6. The operating principle of the trie algorithm.

Fig. 6 illustrates the operating principle of trie. Trie is an extension of a tree structure in which each node stores a character and represents strings through connections between nodes. The nodes corresponding to the first character of every string in the set are connected to the root node, and the ith character is represented as a child node of the $(i-1)$th character. The red nodes denote the ends of each string. The search operation starts from the root node and proceeds by sequentially following the nodes that correspond to the characters of the string. When reaching the end of the string, the value stored in the leaf node is returned.

Trie is highly efficient in terms of time complexity compared with a binary search tree, which is a typical search method. A binary search tree takes an average of $O(logN)$ time to find the desired element. However, when the elements are strings, the time complexity increases to $O(MlogN)$ because comparing the two strings requires additional time proportional to the length of the strings (M is the maximum length of the strings). However, a trie-based string search requires only as much time as the string length. The depth of trie is related to the length of the strings, and since it searches for a single character at each level, it takes $O(M)$ time for a string of maximum length M. Adding a new string to the set also depends directly on the length of the string, taking $O(M)$ time. This is because it follows the nodes corresponding to each character of the string, creating new nodes or utilizing existing

ones. The trie structure is highly effective for string-related operations because its time complexity is linearly proportional to the string length. Adding new words to the dictionary is also simple, allowing quick adaptation to out-of-vocabulary problems. Although there are deep learning-based approaches for handling synonyms, these models cannot immediately adapt to new words, requiring retraining. Therefore, the straightforward dictionary update method of the trie is suitable for real-world scenarios where new words are constantly emerging.

In uniText2SQL, synonyms are elements of a string set, and the actual words used in the database are obtained using the synonyms as keys. *America* and *USA* in the examples of Fig. 4 are stored as elements in the set, and upon searching, they return *the United States*. In the three sentences, *America*, *USA*, and *the United States* are all mapped to the table word *the United States* through the synonym processing module.

3.3. Table selection

In the table selection phase, we identify the tables necessary for SQL query generation. Since existing models refer to a single pre-designated table to derive answers, they have the limitation that the range of answers they can provide is restricted. In contrast, we expand the range of answers that the text-to-SQL system can provide. It explores all tables and selects the most appropriate ones based on reasonable criteria rather than selecting a single table at once. To achieve this, we introduce a table set construction module that supports table selection to select reference tables through a two-step process. Initially, we perform lexicon-based pruning on all tables in the database to form a table set that includes the columns and values mentioned in the question. We then select the top k tables from this set that are semantically most relevant to the question. Since this method applies different criterion at each step of table selection, it allows the question to be considered from various perspectives. Moreover, it offers the flexibility to provide a suitable answer to the question after exploring all tables, while ensuring the potential to find the correct answer as well.

This approach also has advantages well-suited for real-world applications, as it gradually reduces the number of tables based on the criterion applied at each step and selects reference tables. First, the table set construction module significantly reduces the search space for the table selection model, thereby improving efficiency. Large-scale databases are commonly used in real-world environments, and it is inefficient for a deep learning model to search the entire database in real-time for each request. By leveraging a lexicon-based pruning method as an auxiliary module, we substantially enhance the efficiency of the deep learning model. Second, a different criterion is applied at each step, significantly improving table selection performance. Since table selection performance directly impacts query generation performance, improving it is very important. In real-world environments, the content of all tables may not be semantically distinct. Existing table retrieval methods, such as DTR, that rely on semantic similarity may struggle to distinguish similar tables. However, we apply a lexicon matching-based auxiliary module to complement the performance of the table selection model. We utilize a trie algorithm to construct the table set and employ a sentence classification model to select the final reference tables.

When pruning using the trie, we use the table words that appear in the question as keys. The values returned by these keys are the indices of the tables containing columns and values. In this process, we distinguish between the roles of the columns and values. Using a column as a key searches for tables containing that column, and using a value as a key searches for tables containing that specific value.

For example, consider a question *"Which position does Loren Woods play?"*, the sequence labeling model extracts the column *position* and value *Loren Woods*. The extracted words, *position* and *Loren Woods*, are used as keys in the trie to form a table set. These words are sequentially fed into the trie to search for tables containing each word, and the intersection of the searched tables is used as reference tables. Fig. 7 illustrates the pruning process using the trie. Searching with the column *position* as the key returns table indices {1, 2}, and searching with the value *Loren Woods* as the key returns {0, 1, 2}. Consequently, the final table set consists of {1, 2}.

Following the construction of the table set, we select the top k tables most relevant to the question using a sentence classification model. Fig. 8 illustrates the architecture of the table selection model. The model takes a question as input and predicts the table that exhibits the highest relevance to the content of the question. Each table within the database is assigned a class, and the probability of a question being predicted for each class is computed. The model analyzes the context of a question and learns the correlation between the question and the table classes to predict the table class to which the question belongs. However, instead of conducting predictions for all the tables in the database, the table set selected using the trie is used as the prediction range. The equations are as follows.

$$\begin{aligned} S_q &= BERT([CLS]) = PLM(Q) \\ \hat{Y}_t &= W_t S_q + b_t \\ \hat{Y}_t' &= Mask(\hat{Y}_t) \\ P(\hat{Y}_t'|S_q) &= Softmax(\hat{Y}_t') \\ T &= top_k(P(\hat{Y}_t'|S_q)) \end{aligned} \tag{3}$$

S_q denotes the sentence vector for a question obtained from the output of the $[CLS]$ token in BERT. Following the linear transformation of S_q, the resulting vector $\hat{Y}_t$ represents the logits for m table classes. The masking operation involves setting the logits of table classes not included in the table set to $-inf$. The masked logits are then passed through the softmax function and converted into a probability distribution for each table class. T denotes the set of the top k table classes selected from the probability distribution. These k tables are subsequently used as reference tables during query generation.

3.4. SQL query generation

In the SQL query generation phase, we synthesize the information obtained from previous stages to generate the SQL query. Initially, we strengthen schema linking by leveraging the values extracted in the table word extraction phase. We use the values as additional features for question encoding, further reflecting the meaning of the table words. These features clearly emphasize the parts of the question that should be focused on, enabling the model to understand the question based on the meaning of the table words. As a result, an additional connection is established from the question to the tables, inducing bidirectional schema linking. We also expand the scope of query generation by using the k tables selected during the table selection phase. Since existing approaches pre-select a single gold table for query generation, the resulting query can only be executed on that table, completely excluding the possibility of deriving answers from other tables. If a single table is arbitrarily pre-designated, even in the absence of information in the question specifying a particular table, the interpretation of the question can be restricted to that table. Therefore, we use both the question and the k tables as inputs, allowing the model to simultaneously reference multiple tables when generating the query. This opens up the possibility of finding appropriate answers from other tables as well.

Since multiple tables are input simultaneously in the SQL query generation process, a step is required to determine the most suitable table for executing the generated query. If the query is generated by combining columns from different tables, it will not be executable on any table. We select the table to which the column in the SELECT clause belongs from among the k input tables as the final table. During the decoding process, we first predict a column in the SELECT clause, and then restrict the selection range of the columns in the WHERE clause to the table to which the column in the SELECT clause belongs. The reasons we consider determining the optimal table based on this criterion to be the most appropriate are as follows. First, predicting

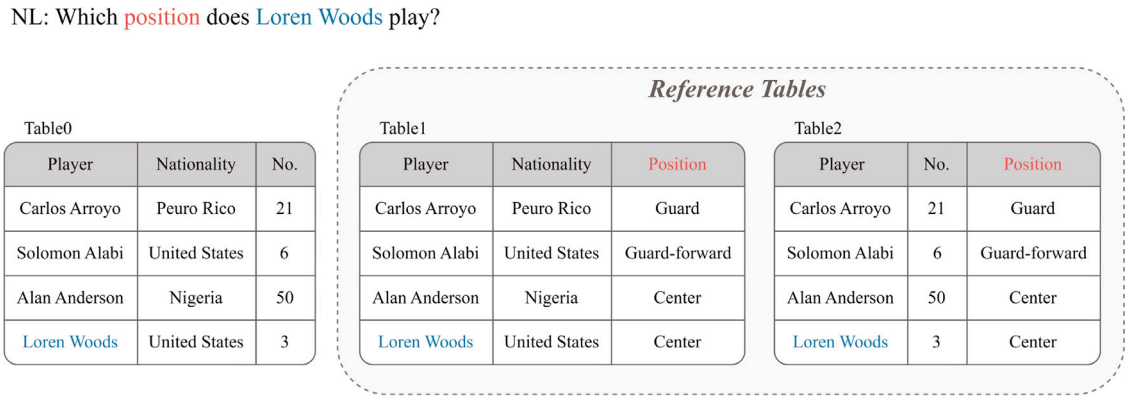

Fig. 7. The pruning process using the trie.

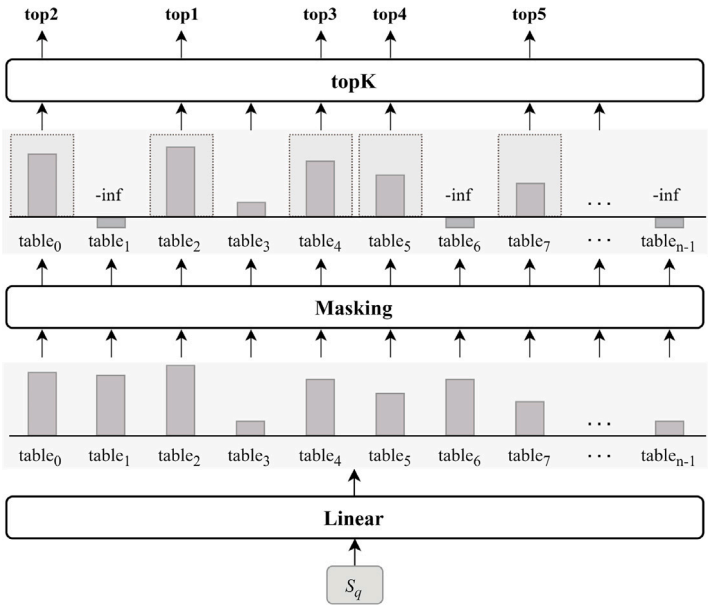

Fig. 8. The architecture of the table selection model.

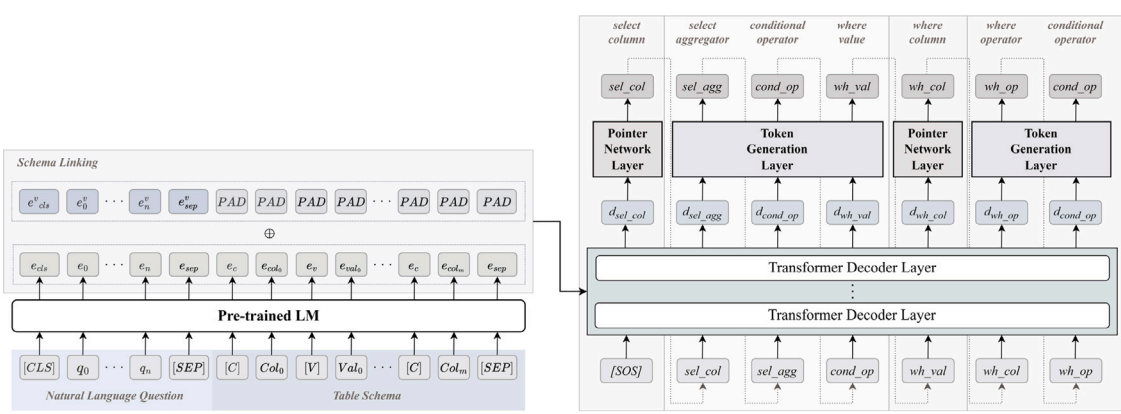

Fig. 9. The architecture of the SQL query generation model.

the column in the SELECT clause is relatively intuitive. The column in the SELECT clause determines the column to be returned as the result, defining the structure of the result set. Thus, the column in the SELECT clause directly reflects the information the user desires and the intent of the question. For example, in the question *"Which position does Loren Woods play?"*, the column *position* in the SELECT clause is directly mentioned, whereas the column *player* in the WHERE clause is not explicitly stated. Therefore, predicting a column in the SELECT clause is relatively easier than predicting columns in the WHERE clause, and it is reasonable to select a table based on this. Second, due to

the syntactic structure of SQL, predicting the SELECT clause first helps to better understand and interpret the consistency of the query. The SELECT clause defines the result set, while the WHERE clause filters it. Therefore, by predicting each element in the typical order of an SQL query, the decoder can more accurately grasp the sequence, role, and context of SQL components. This approach allows for logical consistency in decoding by taking into account the structural characteristics of SQL.

Fig. 9 illustrates the architecture of the SQL query generation model used in uniText2SQL. The SQL query generation model is fundamentally based on a sequence-to-sequence structure, consisting of two parts: an encoder and a decoder. The encoder converts the input sequence into contextualized hidden representations containing the meaning and information of the input sequence, and the decoder uses this information to generate the output sequence. Both effectiveness and efficiency are considered in the practical application of the text-to-SQL system.

3.4.1. Encoder

The encoder receives the question and top k tables as inputs to provide the necessary information for SQL query generation. When dealing with multiple individual schemas, it is crucial to accurately understand the mutual relationship between the question and schemas for effective schema linking. Simultaneously, information between schemas must be maintained independently. In other words, it is important to reflect the mutual linking information between the question and each table, whereas table-to-table connections are not required. Therefore, we use the input format of BRIDGE [21], which enables straightforward and effective schema linking. BRIDGE linearly concatenates the question sequence with a table sequence as the input. Owing to the input length limitation of PLMs, not all table information can be used. Therefore, columns are considered representative information of the schema, and only columns are used as input schema elements. For schema linking, the concept of anchor texts is introduced. Anchor texts refer to values in tables that have high lexical similarity to the question. These values are inserted after the columns to which they belong, thereby inducing schema linking. Following the encoding method of BRIDGE, we use the following input format.

$$\begin{aligned} &X = [CLS], Q, [SEP], T, [SEP] \\ &Q = \{q_0, q_1, q_2, q_3, \dots, q_n\} \\ &T = \{[C], Col_0, [V], Val_0, [C], Col_1, \dots, [C], Col_m\} \end{aligned} \tag{4}$$

The input sequence X is in the form of concatenating the question and the tables, separated by a special token $[SEP]$. Q denotes the tokenized question, and T represents the table sequence in which the tokenized columns are concatenated with a separator token $[C]$. The anchor texts are also tokenized and inserted after the corresponding columns, along with a special token $[V]$. Additionally, we incorporate the value information extracted during the table word extraction phase into the question, as shown in the encoder part of the Fig. 9. This generates embedding vectors for the question with emphasized table words and performs simple yet effective schema linking using only the PLM. The equation for combining values with the question is as follows.

$$\begin{aligned} &E_q = \{e_0, e_1, e_2, e_3, \dots, e_n\} \\ &E_v = \{e_0^v, e_1^v, e_2^v, e_3^v, \dots, e_n^v\} \\ &E_q{}' = E_q + E_v \end{aligned} \tag{5}$$

E_q and E_v represent the embedding vectors of the question and featurized vectors of the values, respectively. The latter is constructed using random embeddings. The result of their element-wise summation, denoted as $E_q{}'$, is utilized in the attention operation with the decoder.

Additionally, we do not use all schema elements from the reference tables. We remove duplicate columns while preserving the order of the reference tables. Fig. 10 illustrates the process of removing duplicate columns from reference tables. The de-duplicated tables pass through BERT in a concatenated form with the question and output embedding vectors reflecting the mutual information between the question and schemas.

We have achieved two functional improvements using this encoding method. The first improvement is in finding anchor texts. Whereas BRIDGE relies on fuzzy string matching between values in the database and the question, we can automatically extract anchor texts from a question using neural network-based sequence labeling, even without field information from the database. The second improvement is schema linking. BRIDGE inserts anchor texts into a schema sequence to induce a connection to a question. In contrast, we construct feature embeddings for the extracted values through sequence labeling, and then add them to the question sequence. These feature embeddings enrich the context of the question and induce bidirectional connections between the question and table sequence, rather than unidirectional connections. In addition to the existing anchor texts that point to crucial schema linking information in the table sequence, we can utilize other anchor texts that point to essential information in the question for schema linking. This indicates the possibility of bidirectional schema linking. This enables the model to correctly identify the values and perform more precise matching between the question and tables. In other words, feature embeddings serve as lexical clues that help the model better understand complex interactions between them.

3.4.2. Decoder

The decoder receives the fused information of the question and the schemas from the encoder to generate an SQL query. We apply the hybrid decoder [39] to generate SQL queries effectively and efficiently. Hybrid decoding combines sequence-based and sketch-based methods, preserving their respective strengths while complementing each other's weaknesses. Sequence-based decoding sequentially predicts each token constituting a query, which is simple and intuitive but prone to syntactic errors depending on the model's understanding of SQL grammar. Sketch-based decoding uses a template that adheres to the SQL syntax as a sketch and performs a type of classification through multiple modules that predict the values for each slot in the sketch. This approach simplifies the complex problem of generating SQL queries into several subtasks, thereby facilitating the production of grammatically correct queries. However, this method is more complex than the sequence-based method because it requires a dedicated module for each slot.

Hybrid decoding combines sketch-based and sequence-based decoding methods. It places the SQL components in each slot of the sketch and sequentially outputs the slot values at each decoding step. This approach predefines the SQL components that can be generated at each step, thereby preventing the grammatical errors that may occur in sequence-based method. Furthermore, by using a single decoder instead of multiple individual modules to predict the slot values, the decoding process becomes relatively simple, and information about each element of the SQL is shared during the prediction stage. A pointer network [40] is applied in the decoding steps to predict the columns in the SELECT and WHERE clauses by selecting columns from the input schema. In the remaining steps for predicting other elements, it generates words with the highest probability from the vocabulary. Once the decoder completes the predictions for all slot values, it synthesizes the generated SQL elements to form a complete executable query. We leverage words extracted from the question for values in the WHERE clause.

3.4.3. Table-constrained decoding

Table-constrained decoding is applied to select the final table on which the generated query will be executed. We select the table to which the column in the SELECT clause belongs from among the k input tables as the final table. During the decoding process, we first predict the column in the SELECT clause and then restrict the prediction of the

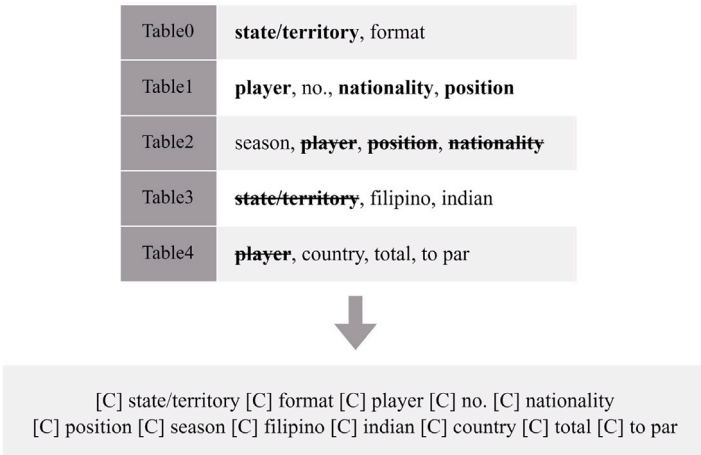

Fig. 10. The process of removing duplicate columns from reference tables.

remaining columns to those from the table to which the column in the SELECT clause belongs. The equation is as follows.

$$E_t = \{e_0^t, e_1^t, e_2^t, e_3^t, \dots, e_p^t\}$$
$$c_{sel} = argmax(\frac{d_{sel_col} \cdot E_t^\top}{\sqrt{|d|}}) \tag{6}$$
$$c_{wh} = argmax(\frac{d_{wh_col} \cdot E_t^\top + M}{\sqrt{|d|}})$$

E_t denotes the embedding vector of the table sequence, while d_{sel_col}, $|d|$, and c_{sel} represent the decoder output at the step of predicting the column in the SELECT clause, the size of the decoder output, and the selected column of the SELECT clause, respectively. c_{sel} is predicted through an attention operation between d_{sel_col} and E_t, where the column with the highest attention score is selected. To predict the column c_{wh} in the WHERE clause, the decoder output d_{wh_col} from the step predicting the column in the WHERE clause is used. We apply a masking matrix $M \in \mathbb{R}^{1\times|E_t|}$ to the attention operation between d_{wh_col} and E_t. For $m_i \in M$, if e_i^t corresponds to a column in the tables containing c_{sel}, m_i is set to 0; otherwise, m_i is set to $-inf$. By adding M to the attention scores between d_{wh_col} and E_t, the prediction range is restricted to the columns in the tables to which c_{sel} belongs, rather than the columns of all tables.

Through the application of table-constrained decoding, and c_{wh} is predicted within the same table as c_{sel}. For example, in Fig. 10, when *position* is predicted as the column in the SELECT clause, the tables containing this column are Tables 1 and 2. When predicting the columns in the WHERE clause, the columns of the other tables are masked, so that one of the columns from these two tables is selected. Consequently, the model predicts one of the columns among *player*, *no.*, *nationality*, *position*, and *season*.

4. Experiments

4.1. Dataset

We conduct experiments on WikiSQL, a large-scale, human-annotated and cross-domain text-to-SQL benchmark. WikiSQL comprises 80,654 pairs of natural language questions and SQL queries along with 24,241 tables extracted from English Wikipedia. Targeting the practical application of text-to-SQL systems, we consider WikiSQL an appropriate dataset for real-world scenarios because it includes tables extracted from the web and realistic questions pertaining to these tables.

To the best of our knowledge, there is no existing dataset suitable for training both the table selection model and the SQL query generation model. With an average of only 3.2 question-SQL query pairs per table, it is challenging to directly use this dataset for training the table selection model. Therefore, we preprocess the WikiSQL dataset to make it suitable for training both table selection and SQL query generation models. First, we merge and refine tables. Merging and refinement are performed according to the following rules.

Table 1
Dataset statistics pre and post preprocessing.

	Original	Preprocessed
# Tables	26,531	10,823
# Question-SQL pairs	80,654	66,497
# Average pairs per table	3.2	4.6

1. Tables without page titles are excluded.
2. When comparing the column compositions of the two tables, the tables are considered for merging only if the columns of one table constitute a subset of those of the other.
3. Among the tables earmarked for merging, only tables with identical page titles are merged.

The page title refers to the title of the Wiki web-page where the table exists. We consider tables with the same page title to cover the same topic and thus include them for merging. Tables without page titles are first excluded. Once the tables for merging are selected, we merge them, taking into account both the page title and table schema.

Subsequently, we preprocess instances. In this step, instances with fewer than two question-SQL query pairs are removed, and the table id and answer column indices of each instance are updated based on the merged table. Finally, preprocessing is conducted for table word extraction. BIO tags are attached to the table elements in the questions for sequence labeling. We pseudo-label table elements that appear in the same form by string-matching the question and the table. The preprocessed dataset consists of 10,823 tables, with 54,019 instances in the training set and 12,476 instances in the test set. Table 1 shows the statistics of the WikiSQL dataset before and after preprocessing.

4.2. Training

We trained the table word extraction and table selection models together using multi-task learning, whereas the SQL query generation model was trained independently. We used the BERT-large-uncased model as the encoder. The hyperparameters used for multi-task learning and training the SQL query generation model are specified in Tables 2 and 3, respectively.

Table 2
Experimental parameter settings for multi-task learning.

Parameter type	Parameter value
Batch size	256
Training epoch	22
Learning rate	3e−4
Dropout	0.3
Loss ratio (multi-task learning)	Table word extraction 0.3 table selection 0.7

Table 3
Experimental parameter settings for SQL query generation model.

Parameter type	Parameter value
Batch size	128
Training epoch	4
Learning rate	1e−4
Dropout	0.3
Decoder hidden size	1024
Transformer decoder layer	8
Attention head	8

4.3. Methods compared

4.3.1. Table selection

BM25 is an algorithm designed to assess the relevance between a query and each document. Based on the concept of bag-of-words, it ranks documents according to the frequency of the query words appearing in each document. We use tables instead of documents as search targets. BM25 is known as a state-of-the-art search algorithm in the TF-IDF family.

DPR† [34] retrieves tables with only a basic understanding of table structure. By leveraging DPR, a general-purpose text-based retriever, it encodes tables in a linearized form and transfer DPR's strong content matching ability to the table retrieval task. This approach outperforms a specialized table-based retriever, DTR, while simplifying the training process and excluding unnecessary complexity.

4.3.2. SQL query generation

SQLova applies six specialized sub-modules for each component of the query. The question and the table are encoded together through a BERT-based table-aware encoder and further contextualized by two bidirectional LSTMs. Each sub-module utilizes this information to predict each component, and the predicted components are synthesized into the final SQL query following a predefined sketch.

X-SQL comprises two encoding layers and an output layer. The first encoding layer utilizes MT-DNN to capture the global context of the question and table. In the second encoding layer, table representations are refined by using the global context representation obtained from the previous layer and type information. The output layer consists of six sub-modules, each of which predicts a component of the SQL query.

HydraNet is composed of multiple sub-modules to predict each component of the query. It uses BERT to encode the question and column pairs, capturing the relationships between the question and each column. Based on the encoded information, the predicted components are synthesized into an SQL query according to rules. It also applies execution-guided decoding to all generated queries to prevent syntax errors.

BRIDGE is based on a BERT-based encoder and a pointer-generator decoder. Two bidirectional LSTMs are applied to a BERT-based encoder. The first bi-LSTM further encodes the question and query, while the second bi-LSTM encodes the metadata of the table. The pointer-generator decoder is based on LSTM, and selective read is applied to extend the input state. BRIDGE achieved state-of-the-art performance on WikiSQL.

Table 4
Performance of table word extraction.

Target	Recall	Precision	F1 score
Value	98.0	97.7	97.9
Column	93.1	90.4	91.7

Table 5
Performance of table selection.

Model	R@1	R@3	R@5	R@10
BM25	31.9	42.9	47.7	54.2
DPR†	31.9	49.1	57.5	68.1
DPR† w/ table set construction	61.3	75.4	80.2	86.3
TS	69.6	81.0	84.4	88.4

4.4. Results

In this section, we present the results for the different parts of uniText2SQL. We analyze and describe the results of the experiments for table word extraction(TE), table selection(TS), and SQL query generation(QG).

4.4.1. Table word extraction

Table 4 presents the extraction performance of table words from natural language questions in the table word extraction phase. Consequently, we achieve F1 scores of 97.9% for the values and 91.7% for the columns, thus demonstrating a high level of extraction accuracy. This suggests that the extracted table words are highly reliable when used as features in the subsequent phases. The lower performance of column extraction compared with value extraction can be attributed to string-match-based pseudo-labeling. Values are primarily proper nouns that appear in almost identical forms in both tables and questions. Most values in the questions can be tagged through string-matching, resulting in a high likelihood of the predicted values matching the actual values. By contrast, columns are often common nouns or phrases, and they frequently appear in different forms between tables and questions. Thus, it is difficult to tag all the columns based on string-matching, leading to instances in which the model predicts incorrect columns. This explains the difference in performance between the value and column extraction.

4.4.2. Table selection

Table 5 presents the results comparing the performance of our table selection model with existing models in finding the tables most suitable for the natural language question, using recall@k as the evaluation metric. In the experiments, the proposed method demonstrates superior performance, showing an average performance difference of approximately 45%p compared with BM25 and 30%p compared with DPR†. Both the proposed method and the other methods show improved performance with an increase in the number of selected tables (k). Although the performance difference is minimal when k is 10, a substantial performance gap of over 34%p and 20%p remain. We speculate that the difficulty in distinguishing tables with similar columns and values is the cause of these results. BM25 is not suitable for distinguishing tables with similar column and value compositions because its searches are based on word frequency. DPR† struggles to identify tables with similar content accurately because it relies solely on the semantic similarity between tables and questions. However, by applying the table set construction module (i.e., lexicon-based pruning), we can effectively identify tables that are difficult to distinguish based solely on a semantic comparison.

To compare the performance of our table selection model and DPR† more fairly, we also perform the table set construction module before performing DPR†. The experimental results show that our model still exhibits an approximately 5%p higher performance. We speculate that this performance gap arises because our table selection model

Table 6
Ablation on design choices of table selection model.

Model	R@1	R@3	R@5	R@10
TS	69.6	81.0	84.4	88.4
w/o table set construction	43.3	56.1	61.3	67.6
w/o multi-task learning	40.0	51.4	56.2	62.0

Table 7
Comparison of training and inference times.

Model	Training time	Inference time
DPR†	130 h 14 m	1.090 s
TS	6 h 49 m	0.169 s

is less influenced by semantic similarity than DPR†. As DPR† uses high-dimensional embeddings to measure similarity, tables with similar content may overlap in the embedding space. In such cases, confusion in the similarity computations may result in reduced search accuracy. By contrast, our table selection model assigns a unique id to each table, making it advantageous for addressing the overlapping issues caused by semantic similarity. Therefore, we consider the proposed model to be more effective in table selection.

We conduct an ablation study to assess the impact of the table word extraction phase on the performance of table selection. Table 6 presents the results of the ablation study on table selection. Initially, we remove the table set construction module and observe a significant performance drop of over 20%p on average. Since we address table selection as a classification problem, reducing the number of classes (i.e., tables) appears to have a significant impact on the performance. Subsequently, we remove multi-task learning with the table word extraction model, which is a basic form of a BERT-based classifier that performs classification using the $[CLS]$ vector. As table words cannot be utilized, lexicon-based pruning is also removed. Compared to when only the table set construction is removed, we observe an additional performance decrease. These results demonstrate that both table set construction and multi-task learning with the table word extraction model are effective at improving the performance of the table selection model. Notably, the table set construction proves to be the most powerful auxiliary tool for table selection.

In general, the training process of deep learning-based retrievers is considerably more complex and time-consuming than that of classification models. By comparing the training and inference times with DPR†, we confirm that the proposed method outperforms DPR† in terms of accuracy and efficiency. Table 7 reports the measurement results for training and inference times. The training time is defined as the time required for the model to converge, whereas the inference time is defined as the time required to process 100 instances. The inference time of the proposed method includes the inference time of the table selection model, as well as the time required for the table set construction module. As a result, the table selection model converges approximately 19 times faster than DPR† and exhibits an inference speed approximately 6 times faster. These experimental results demonstrate the effectiveness and practicality of our table selection model.

4.4.3. SQL query generation

We experimentally determine the number of tables input into the SQL query generation model. Fig. 11(a) illustrates the change in the average and total number of tokens in the input sequence according to the number of input tables, encompassing all the tokens after linearizing the question and table elements. Fig. 11(b) shows the change in the performance of the SQL query generation and execution according to the number of input tables. We measure the performance of SQL query generation using the Exact Match (EM) accuracy, which evaluates whether the set of all clauses in the generated query matches the set of clauses in the gold query. To measure the execution performance of SQL queries, we use the Execution Match (EX) accuracy, which evaluates whether the returned values match when the generated and gold queries are executed.

Fig. 11(a) shows that the number of input tokens increases linearly as the number of tables increases. We observe a gradual increase in the instances in which the number of input tokens exceeds BERT's maximum limit of 512, starting from the point of using three or more tables. In Fig. 11(b), EM increases up to the point where the four tables are input, after which it converges. This is attributed to an increase in noise owing to inaccurate table inputs and, in some cases, the loss of table information owing to exceeding the input length when more than four tables are used. In contrast, EX shows a consistent trend regardless of the number of table inputs. Therefore, using four tables as inputs to the SQL query generation model minimizes the impact of noise and information loss.

To evaluate the performance of uniText2SQL, we compare its SQL query generation performance to that of when a gold table is provided. Table 8 reports the performance comparison results depending on the application of the table selection phase. The abbreviations used to represent each element of the SQL query are listed in Table 9. The performance of uniText2SQL is 83.8% when the gold table is provided, compared to 77.9% otherwise, showing a decrease of approximately 5.9%p. In particular, the column prediction performance decreases significantly by approximately 5.3%p on average, indicating that the overall performance degradation is mainly attributed to the decline in column prediction performance. As our SQL query generation model selects one of the columns in the input sequence through a pointer network, inaccurate inputs of table elements act as noise during column prediction, resulting in performance degradation. EX shows a performance gap of approximately 6.1%p depending on whether the gold table is provided. These experimental results suggest that noise and information loss occurring during the table selection process directly affect the prediction of columns, and that EX exhibits a higher dependence on column prediction than EM.

The table selection performance of the proposed framework is approximately 83%. Despite the propagation of table selection errors to the SQL query generation phase, the decrease in SQL query generation performance is relatively minimal. Specifically, compared to scenarios where the gold table is provided, the query generation performance of the proposed framework maintains approximately 92% for EM and 93% for EX. We speculate that the SQL query generation model effectively mitigates the noise caused by table selection. The schema linking of the proposed model associates only table elements that are highly relevant to the question. This prevents the input tables from acting with equal importance during SQL query generation and minimizes the impact of unnecessary information. This implies that the SQL query generation model can compensate for the weaknesses of the table selection model.

Additionally, we measure the accuracy of the generated SQL queries by referencing tables output from DPR†. Table 10 shows the EM and EX of the generated SQL queries depending on the table selection methods, while Table 11 presents the comparison results of the column prediction accuracy. When generating SQL queries using tables output from DPR†, EM and EX show average performance drops of approximately 53.3%p and 58.0%p, respectively, and the accuracy of the columns decreases by approximately 59.3%p. These experimental results demonstrate that the table selection performance considerably affects SQL query generation. Accurate SQL queries must be generated to ensure the reliability of the data search, which is the purpose of text-to-SQL systems. Referencing accurate tables enables the generation of queries that maintain semantic coherence with the user questions, thereby forming the basis for generating precise queries. Therefore, from a practical perspective, identifying accurate tables is crucial for generating SQL queries.

To better understand the impact of each component of the model on performance, we conduct an ablation study on the SQL query generation model. Table 12 reports the results of the experiments in

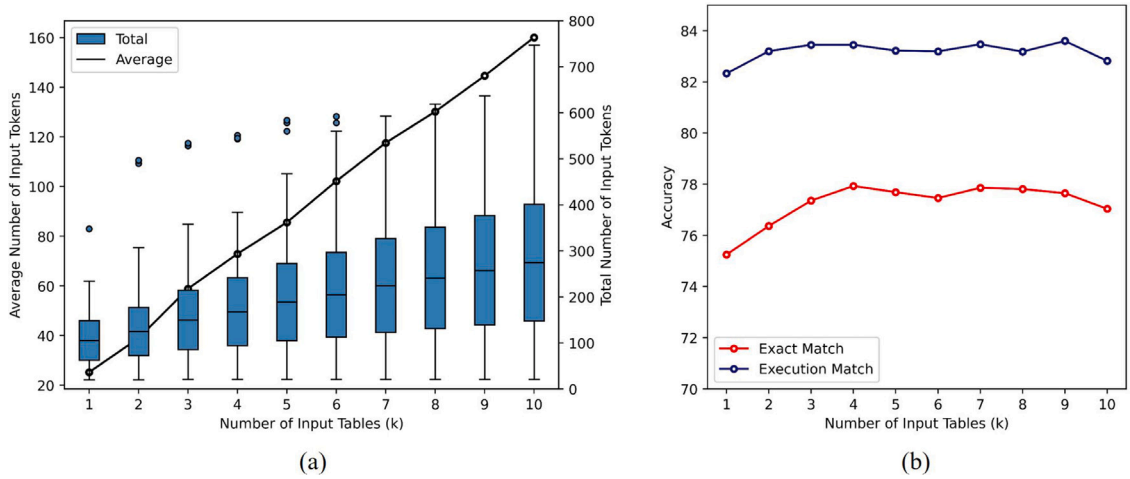

Fig. 11. The change in the input length and the performance according to the number of input tables.

Table 8
Performance of uniText2SQL depending on the application of the table selection phase.

Model	EM	EX	sel_col	sel_agg	sel_cond_op	wh_val	wh_col	wh_comp_op	wh_logic_op
QG w/ gold table	83.8	89.5	97.7	90.8	99.9	97.1	94.9	95.2	97.0
uniText2SQL	77.9	83.4	92.6	90.8	99.9	96.3	89.5	94.9	96.6

Table 9
SQL elements and their descriptions.

Abbreviation	Description
sel_col	Columns in a SELECT clause
sel_agg	Aggregate functions
sel_cond_op	Indicators whether or not to have conditions
wh_val	Values in a WHERE clause
wh_col	Columns in a WHERE clause
wh_comp_op	Comparison operators
wh_logic_op	Logical operators in a WHERE clause

Table 10
SQL query generation performance depending on the table selection methods.

Model	EM/EX (k = 1)	EM/EX (k = 3)	EM/EX (k = 5)	EM/EX (k = 10)
DPR†+QG	21.0/22.0	23.8/25.2	24.6/26.2	24.9/26.5
TS+QG	75.2/82.3	77.4/83.4	77.7/83.2	77.0/82.8

which each component of the proposed model is removed individually. When table word embedding is removed, it exhibits a performance decrease of 0.6%p in EM, and when anchor text is removed, it shows a performance decrease of 2.6%p. Notably, the largest performance drop is observed when both methods are removed. As hypothesized, this demonstrates that the combination of table word embedding and anchor text enables bidirectional schema linking. These two approaches emphasize the parts representing the topics of the user question in both the question and tables, thus providing clear connections between the parts conveying the same information. By enhancing the direct matching between the question and tables, the model can better understand the semantic correlation between them. Consequently, using both methods together, we can leverage the information obtained from each method and achieve bidirectional schema linking through the synergistic effects derived from the interaction between the two methods. Finally, removing table-constrained decoding results in a performance decrease of 0.6%p in EM and a larger decrease of 1.6%p in EX. This is presumed to be because EX has a greater dependency on the column prediction performance. According to the results of the previous experiment, EX showed a greater dependence on the column prediction performance than EM. The pointer network applied for the column prediction specializes in selecting specific positions within the input sequence. The increased number of columns in the input sequence intensifies the challenge of column prediction by necessitating selection from a greater number of columns, consequently diminishing the accuracy. Removing table-constrained decoding increases the complexity of column prediction by requiring consideration of all input tables, thereby exerting a cascading impact on EX.

We also evaluate the performance of our SQL query generation model trained on the original WikiSQL dataset for comparison with existing models. Table 13 presents the results of performance comparisons. Although our SQL query generation model shows lower performance than the state-of-the-art text-to-SQL approach, it achieves satisfactory performance with a relatively simple structure, facilitating the ease of modeling and performance reimplementation in real-word settings. Furthermore, the hybrid decoding method we applied significantly reduces the inference time compared to HydraNet and BRIDGE, which demonstrate similar performances while using a language model of equivalent size. According to the experimental results in Jeong et al. [39], the inference time of HydraNet is 85.2 ms/sentence, and that of BRIDGE is 124.6 ms/sentence. In contrast, the inference time of the hybrid decoding is 71.5 ms/sentence, which is 13.7 ms and 53.1 ms faster than the comparison models, respectively.

HydraNet completes the final SQL query after executing multiple sub-modules and performs execution-guided decoding for all generated queries, resulting in a slow inference speed. In particular, impracticality and inefficiency exist during the training and inference of the column prediction module. As it calculates the probabilities of all column combinations, HydraNet is impractical due to the need for training samples and inefficient because of the increased computational costs during inference. BRIDGE exhibits a substantially slower inference speed because of the sequential processing of the input sequence through the two bi-LSTMs and the methods applied to the decoder to leverage the input sequence information. In addition, it relies heavily on the input table because it has separate layers for encoding the metadata and information of the table. Since both HydraNet and BRIDGE assume an environment in which the gold table is provided, it is challenging to reimplement their performance in scenarios in which the gold table is unavailable for reference.

To compare the applicability in environments where a gold table is not provided, we reimplemented the state-of-the-art model BRIDGE

Table 11
Column prediction accuracy depending on the table selection methods.

Model	sel_col/wh_col (k = 1)	sel_col/wh_col (k = 3)	sel_col/wh_col (k = 5)	sel_col/wh_col (k = 10)
DPR†+QG	30.4/26.9	33.4/29.4	33.8/30.2	33.8/31.0
TS+QG	90.1/87.8	92.6/89.4	92.4/89.4	92.1/89.4

Table 12
Ablation on design choices of the SQL query generation model.

Model	EM	EX
QG	77.9	83.4
w/o table word embeddings	77.3	83.6
w/o anchor texts	75.3	83.3
w/o table word embeddings& anchor texts	74.6	81.4
w/o table-constrained decoding	77.3	81.8

Table 13
Comparison of SQL query generation performance with existing models on the WikiSQL dataset.

Model	Base model	EM	EX
SQLova	BERT-large	80.7	86.2
X-SQL	MT-DNN	83.3	88.7
HydraNet	BERT-large	83.4	88.6
BRIDGE	BERT-large	84.8	90.4
QG	BERT-large	83.9	87.7

Table 14
Comparison of SQL query generation performance depending on the availability of the gold table.

Model		EM
BRIDGE	Gold table	84.6
	TS	72.5
QG	Gold table	83.8
	TS	77.9

and trained it on the preprocessed WikiSQL dataset. As it is designed to take only one table as input, we use the top table output from the table selection model. The results of the comparison between BRIDGE and our SQL query generation model depending on the availability of the gold table are presented in Table 14. In the setting in which the gold table is provided, BRIDGE achieves an EM of 84.6%, which is approximately 0.8%p higher than that of the proposed model. In contrast, when generating queries using the results of table selection as input, the proposed model outperforms BRIDGE by approximately 5.4%p. Depending on the availability of the gold table, BRIDGE exhibits a performance difference of approximately 12.1%p, whereas our model shows a difference of only approximately 5.9%p. These results indicate that the structure of BRIDGE is highly dependent on the input table, suggesting a lower applicability in real-world scenarios where the gold table cannot be referenced. Additionally, since BRIDGE performs fuzzy string matching between table words and questions to find anchor texts, in environments where field information is not provided, it is expected that the query generation performance will drop further because of the inability to use anchor texts.

We experimentally demonstrate the applicability of our SQL query generation model to real-world scenarios. The proposed model exhibits fast inference speed with its relatively lightweight encoder and efficient hybrid decoding and mitigates the performance limitations of the previous phase by reducing the dependence on input table. It also has a structure that enables it to process multiple tables and extract anchor texts from the questions. Therefore, it is considered most suitable for practical applications in text-to-SQL systems.

Table 15 presents examples of queries generated by uniText2SQL. The generation cases show the SQL queries generated by referencing the tables selected through the table selection model and the corresponding answers obtained by executing each query. In real-world scenarios, as it is unclear which table information a user desires, queries must be generated based solely on the user questions. Existing systems cannot generate queries solely based on user questions because they are trained to use table information to generate queries. However, uniText2SQL can generate queries by selecting all possible tables based on the question. These examples demonstrate that uniText2SQL can provide various results derived from multiple tables. For the question in the first example, multiple tables are available depending on the year. As the question of which year's information the user wants is ambiguous, uniText2SQL considers all possible tables to generate queries. In each generation, the answer to the question is searched based on the NHL Entry Drafts held in 1994, 1983, 1995, and 1972, respectively. In the second example, the criterion for table selection is tournament, and no specific tournament is specified in the question. In each generation case, the answer to the question is searched based on tournaments named Credit Union Challenge and Northeast Delta International, respectively. The proposed framework can satisfy users by providing diverse answers by referencing multiple tables selected based on reasonable criteria.

Table 16 illustrates examples of SQL queries generated by uniText2SQL and their execution results depending on whether the synonym processing module is applied. Each example consists of a question containing a synonym, gold SQL query, SQL query generated by applying the synonym processing module to map synonyms to words in the table, and SQL query generated without handling synonyms. In the first example, 'w/o synonym processing' shows the issue of constructing incorrect reference tables and producing incorrect queries and results due to the absence of synonym replacement with table words. The synonym used in the question is *Holland*, while the database stores *the Netherlands*. When *Holland* is used to construct reference tables, tables containing *Holland* instead of *the Netherlands* are selected. Queries generated by referencing incorrect tables ultimately yield answers irrelevant to the user question. In contrast, 'w/ synonym processing' shows the correct result by handling the synonym.

The second example illustrates the problem of generating queries without handling synonyms, resulting in a query that fails to retrieve results from the database. The synonym used in the question is *possessor*, while *owner* is stored in the database. We construct a table set that includes the table words extracted during the table word extraction phase and use this set as the prediction range for the table selection phase. Since there are no other tables in the database with a *possessor* column, the *possessor* column does not influence the construction of reference tables. However, as no table in the database contains a *possessor* column, the final query cannot be executed. These problems may arise because users lack prior knowledge of databases. Therefore, a step to handle synonyms is necessary for the practical use of text-to-SQL systems.

5. Conclusion

We propose a unified all-in-one framework called uniText2SQL for effectively utilizing text-to-SQL systems in real-world scenarios. Existing text-to-SQL research assumes an environment in which a table containing answers is provided. Although this premise enables the effective generation of complex and challenging SQL queries by leveraging information from the gold table, existing models face limitations in their applicability to real-world environments when a gold table is not provided. Therefore, by adding phases to identify tables related to a natural language question, uniText2SQL consists of three phases:

Table 15
Examples of SQL queries generated by uniText2SQL.

Question	The nhl team new york islanders is what nationality?
Generation 1	SQL: SELECT nationality FROM 1994 NHL Entry Draft WHERE nhl team = 'new york islander' Answer: Canada
Generation 2	SQL: SELECT nationality FROM 1983 NHL Entry Draft WHERE nhl team = 'new york islanders' Answer: Canada
Generation 3	SQL: SELECT nationality FROM 1995 NHL Entry Draft WHERE nhl team = 'new york islanders' Answer: Czech republic
Generation 4	SQL: SELECT nationality FROM 1972 NHL Amateur Draft WHERE nhl team = 'new york islanders' Answer: Canada
Question	When are all years that the champion is ji min jeong?
Generation 1	SQL: SELECT year FROM Credit Union Challenge WHERE champion = 'ji min jeong' Answer: 2006
Generation 2	SQL: SELECT year FROM Northeast Delta Dental International WHERE champion = 'ji min jeong' Answer: 2007

Table 16
Examples of SQL queries generated by uniText2SQL depending on the application of the synonym processing module.

Question	Name the total number of round for against *Holland*.
Gold	SQL: SELECT count (round) FROM Rafael Nadal career statistics WHERE against = *'the Netherlands'* Answer: 1
w/ synonym processing	SQL: SELECT count (round) FROM Rafael Nadal career statistics WHERE against = *'the Netherlands'* Answer: 1
w/o synonym processing	SQL: SELECT count (round) FROM 1981 Oakland Raiders season Answer: 9
Question	'How many age entries had a weight of 10-7 and an *possessor* of Double Octagon Partnership?
Gold	SQL: SELECT count (age) FROM 2006 Grand National WHERE weight (st, lb) = '10-7' AND *owner* = 'double octagon partnership' Answer: 1
w/ synonym processing	SQL: SELECT count (age) FROM 2006 Grand National WHERE weight (st, lb) = '10–7' AND *owner* = 'double octagon partnership' Answer: 1
w/o synonym processing	SQL: SELECT count (age) FROM 2006 Grand National WHERE weight (st, lb) = '10-7' AND ' ' = 'double octagon partnership' Answer: Inexecutable

table word extraction, table selection, and SQL query generation. Consequently, uniText2SQL enables a query-based search for structured data, demonstrating high utility in real-world applications. However, we target simple and intuitive queries, such as WikiSQL, and reduce the complexity of the model to enhance the practicality of text-to-SQL systems. This design may lead to a lack of sub-querying capability for understanding the relationships between tables and generating complex queries. Therefore, in the future, we aim to develop a practical text-to-SQL system capable of handling complex queries.

CRediT authorship contribution statement

Mirae Han: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Data curation, Conceptualization. **Seongsik Park:** Writing – review & editing, Methodology, Conceptualization. **Seulgi Kim:** Validation, Investigation. **Harksoo Kim:** Supervision, Project administration, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This paper was supported by Konkuk University, Republic of Korea in 2023.

Data availability

Data will be made available on request.

References

[1] V. Zhong, C. Xiong, R. Socher, Seq2sql: Generating structured queries from natural language using reinforcement learning, 2017, arXiv preprint arXiv:1709.00103.

[2] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, et al., Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task, in: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, 2018, pp. 3911–3921.

[3] R. Giaquinto, D. Zhang, B. Kleiner, Y. Li, M. Tan, P. Bhatia, R. Nallapati, X. Ma, Multitask pretraining with structured knowledge for text-to-sql generation, in: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2023, pp. 11067–11083.

[4] X. Xu, C. Liu, D. Song, Sqlnet: Generating structured queries from natural language without reinforcement learning, 2017, arXiv preprint arXiv:1711.04436.

[5] T. Yu, Z. Li, Z. Zhang, R. Zhang, D. Radev, TypeSQL: Knowledge-based type-aware neural text-to-SQL generation, in: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers), 2018, pp. 588–594.

[6] L. Dong, M. Lapata, Coarse-to-fine decoding for neural semantic parsing, in: Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2018, pp. 731–742.

[7] C. Wang, K. Tatwawadi, M. Brockschmidt, P.-S. Huang, Y. Mao, O. Polozov, R. Singh, Robust text-to-sql generation with execution-guided decoding, 2018, arXiv preprint arXiv:1807.03100.

[8] B. Hui, X. Shi, R. Geng, B. Li, Y. Li, J. Sun, X. Zhu, Improving text-to-sql with schema dependency learning, 2021, arXiv preprint arXiv:2103.04399.

[9] W. Hwang, J. Yim, S. Park, M. Seo, A comprehensive exploration on wikisql with table-aware word contextualization, 2019, arXiv preprint arXiv:1902.01069.

[10] P. He, Y. Mao, K. Chakrabarti, W. Chen, X-SQL: reinforce context into schema representation, Microsoft Res.: Artif. Intell. (2019).

[11] Q. Lyu, K. Chakrabarti, S. Hathi, S. Kundu, J. Zhang, Z. Chen, Hybrid ranking network for text-to-sql, 2020, arXiv preprint arXiv:2008.04759.

[12] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: Pre-training of deep bidirectional transformers for language understanding, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), 2019, pp. 4171–4186.

[13] X. Liu, P. He, W. Chen, J. Gao, Multi-task deep neural networks for natural language understanding, in: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, 2019, pp. 4487–4496.

[14] T. Yu, M. Yasunaga, K. Yang, R. Zhang, D. Wang, Z. Li, D. Radev, SyntaxSQLNet: Syntax tree networks for complex and cross-domain text-to-SQL task, in: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, 2018, pp. 1653–1663.

[15] O. Rubin, J. Berant, SmBoP: Semi-autoregressive bottom-up semantic parsing, in: Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2021, pp. 311–324.

[16] H. Li, J. Zhang, C. Li, H. Chen, Resdsql: Decoupling schema linking and skeleton parsing for text-to-sql, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 37, 2023, pp. 13067–13075.

[17] D. Jha, L. Ward, Z. Yang, C. Wolverton, I. Foster, W.-k. Liao, A. Choudhary, A. Agrawal, IRNet: A general purpose deep residual regression framework for materials discovery, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2019, pp. 2385–2393.

[18] Y. Gan, X. Chen, J. Xie, M. Purver, J.R. Woodward, J. Drake, Q. Zhang, Natural SQL: Making SQL easier to infer from natural language specifications, in: Findings of the Association for Computational Linguistics: EMNLP 2021, 2021, pp. 2030–2042.

[19] Z. Dong, S. Sun, H. Liu, J.-G. Lou, D. Zhang, Data-anonymous encoding for text-to-SQL generation, in: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP, 2019, pp. 5405–5414.

[20] W. Lei, W. Wang, Z. Ma, T. Gan, W. Lu, M.-Y. Kan, T.-S. Chua, Re-examining the role of schema linking in text-to-SQL, in: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP, 2020, pp. 6943–6954.

[21] X.V. Lin, R. Socher, C. Xiong, Bridging textual and tabular data for cross-domain text-to-SQL semantic parsing, in: Findings of the Association for Computational Linguistics: EMNLP 2020, 2020, pp. 4870–4888.

[22] B. Bogin, M. Gardner, J. Berant, Global reasoning over database structures for text-to-SQL parsing, in: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP, 2019, pp. 3659–3664.

[23] B. Wang, R. Shin, X. Liu, O. Polozov, M. Richardson, RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers, in: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, 2020, pp. 7567–7578.

[24] R. Cao, L. Chen, Z. Chen, Y. Zhao, S. Zhu, K. Yu, LGESQL: line graph enhanced text-to-SQL model with mixed local and non-local relations, 2021, arXiv preprint arXiv:2106.01093.

[25] R. Cai, J. Yuan, B. Xu, Z. Hao, Sadga: Structure-aware dual graph aggregation network for text-to-sql, Adv. Neural Inf. Process. Syst. 34 (2021) 7664–7676.

[26] B. Bogin, J. Berant, M. Gardner, Representing schema structure with graph neural networks for text-to-SQL parsing, in: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, 2019, pp. 4560–4565.

[27] Y. Li, R. Zemel, M. Brockschmidt, D. Tarlow, Gated graph sequence neural networks, in: Proceedings of ICLR'16, 2016.

[28] S. Robertson, Understanding inverse document frequency: on theoretical arguments for IDF, J. Doc. 60 (5) (2004) 503–520.

[29] S. Robertson, H. Zaragoza, The probabilistic relevance framework: BM25 and beyond, Found. Trends Inf. Retr. 3 (4) (2009) 333–389.

[30] A. Vaswani, Attention is all you need, Adv. Neural Inf. Process. Syst. (2017).

[31] P.-S. Huang, X. He, J. Gao, L. Deng, A. Acero, L. Heck, Learning deep structured semantic models for web search using clickthrough data, in: Proceedings of the 22nd ACM International Conference on Information & Knowledge Management, 2013, pp. 2333–2338.

[32] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, W.-t. Yih, Dense passage retrieval for open-domain question answering, in: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP, 2020, pp. 6769–6781.

[33] J. Herzig, T. Mueller, S. Krichene, J. Eisenschlos, Open domain question answering over tables via dense retrieval, in: Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2021, pp. 512–519.

[34] Z. Wang, Z. Jiang, E. Nyberg, G. Neubig, Table retrieval may not necessitate table-specific model design, in: Proceedings of the Workshop on Structured and Unstructured Knowledge Integration, SUKI, 2022, pp. 36–46.

[35] E.M. Voorhees, et al., The trec-8 question answering track report., in: Trec, Vol. 99, 1999, pp. 77–82.

[36] H. Chen, X. Liu, D. Yin, J. Tang, A survey on dialogue systems: Recent advances and new frontiers, Acm Sigkdd Explor. Newsl. 19 (2) (2017) 25–35.

[37] Z. Ji, Z. Lu, H. Li, An information retrieval approach to short text conversation, 2014, arXiv preprint arXiv:1408.6988.

[38] B. Kostić, J. Risch, T. Möller, Multi-modal retrieval of tables and texts using tri-encoder models, in: Proceedings of the 3rd Workshop on Machine Reading for Question Answering, 2021, pp. 82–91.

[39] G. Jeong, M. Han, S. Kim, Y. Lee, J. Lee, S. Park, H. Kim, Improving text-to-SQL with a hybrid decoding method, Entropy 25 (3) (2023) 513.

[40] O. Vinyals, M. Fortunato, N. Jaitly, Pointer networks, Adv. Neural Inf. Process. Syst. 28 (2015).